Université d'El Oued

Faculté des Sciences Exactes

Département d'Informatique

Compte rendu de travaux pratiques

TP05

Real-Time Data Streaming and Processing Using Apache Kafka

Réalisé par : Amina Kenioua

Année universitaire 2026/2027

1 Objectif

يهدف هذا العمل التطبيقي إلى فهم كيفية استخدام Kafka Apache في بث البيانات ومعالجتها في الوقت الحقيقي.

وتتمثل الأهداف الرئيسية لهذا العمل فيما يلي:

- فهم دور Kafka Apache في أنظمة بث البيانات الموزعة.
- التعرف على المفاهيم الأساسية: Broker وTopic وPartition وProducer وConsumer وConsumer Group.
- تثبيت وتشغيل Kafka باستخدام Compose Docker في وضع KRaft.
- تطوير برنامج Producer باستخدام Python لإنشاء بيانات المستشعرات وإرسالها.
- تطوير برنامج Consumer لاستقبال البيانات وتحليلها في الوقت الحقيقي.
- اكتشاف حالات ارتفاع درجة الحرارة وإصدار تنبيهات عند حدوثها.

2 Outils utilisés

تم استخدام الأدوات والتقنيات التالية لإنجاز هذا العمل التطبيقي:

- **Desktop: Docker** لتشغيل Kafka داخل حاوية.
- **Compose: Docker** لإعداد Kafka وإدارة تشغيلها.
- **3.9.1 Kafka Apache:** لإدارة الرسائل وبث البيانات.
- **3.13 Python:** لتطوير برنامجي Producer وConsumer.
- **2.3.3 kafka-python:** مكتبة Python للتواصل مع Kafka.
- **Nano: / Code Studio Visual** لإنشاء الملفات وتعديل الأكواد.
- **Terminal:** لتنفيذ الأوامر ومراقبة النظام.
- **GitHub:** لحفظ المشروع ومشاركة ملفاته.

3 Étapes de réalisation

3.1 Étape 1 : Création du répertoire du projet

تم إنشاء مجلد باسم `tp05-kafka` لتنظيم ملفات المشروع، ويحتوي على الملفات التالية:

```
tp05-kafka/
|-- compose.yaml
|-- requirements.txt
|-- producer.py
|-- consumer.py
|-- README.md
|-- screenshots/
```

3.2 Étape 2 : Lancement de Kafka avec Docker Compose

تم إنشاء ملف باسم `compose.yaml` لإعداد وتشغيل خادم Kafka واحد باستخدام وضع KRaft، دون الحاجة إلى استخدام ZooKeeper.

بعد إعداد الملف، تم تشغيل Kafka باستخدام الأمر التالي:

```
docker compose up -d
```

وللتحقق من حالة الحاوية والتأكد من تشغيلها، تم تنفيذ الأمر:

```
docker compose ps
```

3.3 Étape 3 : Installation de la bibliothèque Python Kafka

تم إنشاء بيئة افتراضية لعزل مكتبات المشروع عن مكتبات Python الأساسية، وذلك باستخدام الأمرين التاليين:

```
python3 -m venv .venv
source .venv/bin/activate
```

بعد ذلك، تم تثبيت المكتبة المطلوبة للتواصل مع Kafka:

```
python -m pip install -r requirements.txt
```

تم استخدام الإصدار `kafka-python 2.3.3` بعد مواجهة مشكلة توافق مع الإصدار السابق عند استعمال Python 3.13.

3.4 Étape 4 : Création d'un Topic

تم إنشاء Topic باسم `campus-sensors` داخل Kafka، وهو المسؤول عن استقبال وتنظيم رسائل بيانات المستشعرات.

وقد تم اعتماد الإعدادات التالية:

- اسم Topic: `campus-sensors`.
- عدد Partitions: 3.
- Replication Factor: 1.

يسمح هذا التنظيم بتقسيم الرسائل إلى أجزاء، مما يساعد على توزيع البيانات ومعالجتها.

3.5 Étape 5 : Création du Producer

تم تطوير البرنامج `producer.py` لإنشاء وإرسال 30 حدثاً افتراضياً يمثل بيانات مستشعرات داخل الحرم الجامعي.
يحتوي كل حدث على المعلومات التالية:

- ID: Event معرّف الحدث.
- ID: Sensor معرّف المستشعر.
- Temperature: درجة الحرارة.
- Humidity: نسبة الرطوبة.
- Timestamp: وقت تسجيل البيانات.

يقوم Producer بإرسال الأحداث إلى Topic المسمى `campus-sensors` بصيغة JSON، بمعدل حدث واحد في الثانية.

3.6 Étape 6 : Création du Consumer

تم تطوير البرنامج `consumer.py` لاستقبال الرسائل المرسلة إلى Kafka وقراءتها وتحليلها.
وتتمثل المهام الرئيسية لهذا البرنامج فيما يلي:

- استقبال الرسائل من Topic.
- تحويل بيانات JSON إلى كائنات يمكن التعامل معها في Python.
- عرض درجة الحرارة والرطوبة والتوقيت.
- مراقبة قيم درجة الحرارة واكتشاف القيم المرتفعة.

عندما تكون درجة الحرارة أكبر من أو تساوي 35 درجة مئوية، يعرض البرنامج التنبيه التالي:

```
ALERT: High temperature detected!
```

3.7 Étape 7 : Test du système

لاختبار النظام، تم تشغيل Consumer أولاً في نافذة Terminal، ثم تشغيل Producer في نافذة Terminal ثانية.

بعد ذلك، بدأ Producer بإرسال أحداث المستشعرات إلى Kafka، بينما قام Consumer باستقبال الرسائل وعرض بياناتها في الوقت الحقيقي.

يسمح هذا الاختبار بالتحقق من عملية الاتصال بين Producer وKafka وConsumer، وكذلك التأكد من إمكانية اكتشاف درجات الحرارة المرتفعة وإظهار التنبيهات.

4 Réponses aux questions du compte rendu

4.1 Question 1 : Quel problème Apache Kafka résout-il dans une architecture de streaming de données ?

يحل Kafka Apache مشكلة جمع ونقل وتوزيع البيانات في الوقت الحقيقي، خاصة عندما يتم إنشاء كميات كبيرة من البيانات من مصادر مختلفة.

يسمح Kafka للتطبيقات المختلفة بتبادل الأحداث والرسائل بطريقة منظمة وموثوقة، دون الحاجة إلى اتصال مباشر بين جميع التطبيقات.

وبالتالي، يساعد على بناء أنظمة موزعة قادرة على استقبال البيانات ومعالجتها بشكل مستمر.

4.2 Question 2 : Définir Producer, Consumer, Broker, Topic, Partition, Offset et Consumer Group.

- **Producer:** هو البرنامج أو التطبيق الذي يقوم بإرسال الرسائل والبيانات إلى Kafka.
- **Consumer:** هو البرنامج الذي يقوم باستقبال وقراءة الرسائل من Kafka.
- **Broker:** هو خادم Kafka المسؤول عن تخزين الرسائل وإدارتها.
- **Topic:** هو تصنيف منطقي يتم استخدامه لتنظيم الرسائل حسب نوعها أو موضوعها.
- **Partition:** هو جزء من Topic يتم فيه تخزين الرسائل، ويسمح بتوزيع البيانات ومعالجتها بالتوازي.
- **Offset:** هو رقم يحدد موقع رسالة معينة داخل Partition.
- **Group: Consumer** هي مجموعة من المستهلكين تعمل معاً لقراءة الرسائل وتوزيع عملية الاستهلاك بينها.

4.3 Question 3 : Quelle est la différence entre un Topic et une Partition dans Kafka ?

الفرق بين Topic وPartition هو أن Topic يمثل حاوية منطقية تُستخدم لتنظيم الرسائل حسب نوع البيانات أو موضوعها.

أما Partition فهو تقسيم داخلي لهذا Topic، حيث يتم تخزين الرسائل داخله بشكل مرتب.

يمكن أن يحتوي Topic واحد على عدة Partitions، وهذا يسمح بتوزيع الرسائل وتحسين قابلية التوسع والمعالجة المتوازية.

4.4 Question 4 : Pourquoi un Consumer utilise-t-il `auto_offset_reset='earl`

يُستخدم الخيار `auto_offset_reset='earliest'` لجعل Consumer يبدأ قراءة الرسائل من أقدم رسالة متاحة داخل Partition، وذلك عندما لا يكون هناك Offset محفوظ وصالح لمجموعة المستهلكين.

يكون هذا الخيار مفيداً عند تشغيل Consumer لأول مرة، لأنه يسمح له بقراءة الرسائل القديمة المتوفرة بدلاً من البدء من أحدث رسالة فقط.

ولا يعيد هذا الخيار قراءة جميع الرسائل في كل مرة، إذا كان هناك Offset محفوظ وصالح مسبقاً.

4.5 Question 5 : Quel est le rôle du Group ID d'un Consumer ? Que se passe-t-il lorsqu'un nouveau Group ID est utilisé ?

يُستخدم ID Group لتحديد مجموعة المستهلكين التي ينتمي إليها Consumer، كما يسمح لـ Kafka بإدارة مواقع قراءة الرسائل (Offsets) الخاصة بهذه المجموعة.

عند استخدام ID Group جديد، يعتبر Kafka أن هناك مجموعة مستهلكين جديدة.

إذا لم تكن لهذه المجموعة Offsets محفوظة، وكان الخيار `auto_offset_reset='earliest'` مفعلاً، فإن Consumer يستطيع البدء في قراءة الرسائل المتاحة من أقدم رسالة.

4.6 Question 6 : Pourquoi le format JSON est-il adapté à ce travail pratique ? Mentionner un inconvénient.

يُعتبر JSON مناسباً لهذا العمل التطبيقي لأنه تنسيق بسيط وسهل القراءة والفهم، كما أنه مدعوم من طرف العديد من لغات البرمجة.

يسمح بتنظيم بيانات المستشعرات في شكل حقول واضحة، مثل درجة الحرارة والرطوبة والتوقيت، مما يسهل إرسالها واستقبالها ومعالجتها باستخدام Python.

أما من عيوبه، فقد يكون حجم بيانات JSON أكبر من بعض التنسيقات الثنائية (Binary Formats)، مما قد يؤدي إلى استهلاك مساحة تخزين وعرض نطاق أكبر.

4.7 Question 7 : Comment Docker Compose simplifie-t-il le déploiement par rapport au lancement manuel des services ?

يسهّل Compose Docker عملية تشغيل وإدارة الخدمات من خلال وضع إعداداتها داخل ملف واحد يسمى `compose.yaml`.

بدلاً من إعداد وتشغيل كل خدمة بشكل منفصل، يمكن تشغيل البيئة كاملة باستخدام أمر واحد:

```
docker compose up -d
```

وهذا يجعل عملية النشر أكثر سهولة وتنظيماً، كما يسمح بإعادة تشغيل نفس البيئة دون الحاجة إلى إعادة إعداد جميع الخدمات يدوياً.

4.8 Question 8 : Quelles sont les limites d'une configuration éducative avec un seul Broker par rapport à un cluster de production ?

تحتوي هذه البيئة التعليمية التي تعتمد على Broker واحد على عدة محدوديات، من أهمها:

- استخدام خادم Kafka واحد فقط.
- عدم توفير تحمل الأعطال على مستوى الخوادم (Fault Tolerance).
- محدودية قابلية التوسع عند زيادة حجم البيانات.
- استخدام Factor Replication يساوي 1، أي عدم وجود نسخ مكررة للبيانات على خوادم أخرى.
- عدم توفير مستوى التوافر العالي (High Availability) المطلوب في أنظمة الإنتاج.

لذلك، تعتبر هذه الإعدادات مناسبة للتعلم والتجارب، لكنها ليست كافية لتشغيل أنظمة إنتاج كبيرة تتطلب الاستمرارية وتحمل الأعطال.

4.9 Question 9 : Fournir des preuves montrant le conteneur en cours d'exécution, la description du Topic, ainsi que les résultats du Producer et du Consumer.

يمكن إثبات تنفيذ هذا العمل التطبيقي من خلال النتائج التالية:

1. نتيجة الأمر `docker compose ps` التي توضح حالة حاوية Kafka.
2. نتيجة وصف Topic المسمى `campus-sensors`، والتي تعرض عدد Partitions وReplication Factor.

3. مخرجات Producer التي توضح إنشاء وإرسال أحداث المستشعرات.

4. مخرجات Consumer التي توضح استقبال الرسائل وعرض البيانات والتنبيهات عند ارتفاع درجة الحرارة.

تُستخدم هذه النتائج للتحقق من أن مكونات النظام تعمل معاً، وأن عملية إرسال واستقبال البيانات تتم عبر Kafka.

5 Problèmes rencontrés et solutions

5.1 Problème 1 : Erreur d'importation de la bibliothèque Kafka

أثناء استخدام الإصدار `kafka-python 2.0.2` مع Python 3.13، ظهرت مشكلة عند استيراد مكتبة Kafka، وكانت رسالة الخطأ كما يلي:

```
ModuleNotFoundError:
No module named 'kafka.vendor.six.moves'
```

Solution :

لحل هذه المشكلة، تم حذف الإصدار القديم من المكتبة وتثبيت الإصدار 2.3.3، وذلك باستخدام الأوامر التالية:

```
python -m pip uninstall kafka-python -y
python -m pip install -r requirements.txt
```

بعد تثبيت الإصدار الجديد، تم اختبار عملية الاستيراد بنجاح، وظهرت الرسالة التالية:

```
Kafka library installed successfully
```

5.2 Problème 2 : Organisation des bibliothèques Python

لتجنب الخلط بين مكتبات المشروع ومكتبات Python الأساسية، تم إنشاء بيئة افتراضية خاصة بالمشروع. وقد تم تنفيذ الأوامر التالية:

```
python3 -m venv .venv
source .venv/bin/activate
```

بعد تفعيل البيئة الافتراضية، تم تثبيت المكتبات المطلوبة داخلها، مما يساعد على تنظيم المشروع وتجنب مشاكل توافق المكتبات.

6 Conclusion

في هذا العمل التطبيقي، قمنا بتثبيت وتشغيل Kafka Apache باستخدام Compose Docker في وضع KRaft.

كما قمنا بتطوير برنامج Producer لإرسال بيانات المستشعرات، وبرنامج Consumer لاستقبال هذه البيانات وتحليلها في الوقت الحقيقي.

وتعرفنا أيضاً على المفاهيم الأساسية في Kafka، مثل Topic وPartition وBroker وConsumer Group وOffset.

بالإضافة إلى ذلك، قمنا بإضافة آلية لمراقبة درجة الحرارة وإظهار تنبيه عندما تكون درجة الحرارة أكبر من أو تساوي 35 درجة مئوية.

من خلال هذا العمل، تعرفنا على أهمية Kafka Apache في بناء أنظمة بث البيانات ومعالجتها في الوقت الحقيقي، مثل أنظمة مراقبة الحرم الجامعي الذكي.